

PROJECT REPORT

**“StiyaOS – A browser-based Operating System
Simulation using React and Modern
Web Technologies”**

Report submitted for the partial fulfilment of the requirement for the
DIPLOMA IN *COMPUTER SCIENCE & ENGINEERING (CSE), 6th SEMESTER*
(Final Year)



**MAHAMAYA POLYTECHNIC OF INFORMATION TECHNOLOGY
SALEMPUR HATHRAS (U.P.) - 204102**

Board of Technical Education Uttar Pradesh

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(SESSION – 2023 TO 2026)

Submitted by:

Harsh Kumar Singh

Enrolment No: E23113735500026

SUBMITTED TO:

UNDER THE GUIDANCE OF:

CERTIFICATE

This is to certify that **Mr. Harsh Kumar Singh**, bearing Enrolment No. **E23113735500026**, is a Bonafide student of **Diploma in Computer Science and Engineering** at **Mahamaya Polytechnic of Information Technology, Salempur, Hathras (U.P.) - 204102**.

He has successfully completed the Major Project work entitled "**StiyaOS: A Browser-Based Operating System Simulation Using React and Modern Web Technologies**" during the academic session *2023-2026* under the prescribed curriculum of the Board of Technical Education, Uttar Pradesh.

The project report submitted by him is an original work carried out under proper guidance and supervision. To the best of our knowledge, this work has not been submitted previously, either in part or in full, for the award of any other degree or diploma.

This report is submitted in partial fulfilment of the requirements for the award of the Diploma in Computer Science and Engineering under the Board of Technical Education, Uttar Pradesh.

.....
Signature of Internal Guide

.....
Signature of HOD

ACKNOWLEDGEMENT

I am grateful to Mahamaya Polytechnic of Information Technology, Salempur Hathras (U.P.) for providing the academic environment and infrastructure that supported me during the preparation of this project report titled "StiyaOS – A browser-based Operating System Simulation Using React and Modern Web Technologies"

I acknowledge the Department of Computer Science & Engineering for creating a platform that encourages students to work independently and explore new technologies on their own. The exposure provided by the institute helped me gain confidence and motivated me to develop this project with sincerity and dedication.

I would also like to acknowledge the organization where I completed my Full Stack Development training, which helped me gain practical exposure.

I express my deep gratitude to my family for their unwavering support, inspiration, and encouragement, which has always been a source of strength for me.

Lastly, I take full responsibility for any errors, omissions, or inaccuracies that may appear in this report, as the development and writing have been carried out with my own best efforts and understanding.

Harsh Kumar Singh

Enrolment No: E23113735500026

Diploma CSE, Session 2023–2026

ABSTRACT

StiyaOS is a browser-based Operating System simulation developed to provide an operating system-like experience directly within a web browser. The project aims to bridge the gap between traditional desktop operating systems and modern web applications by offering a unified platform where users can access and manage various applications without the need for local installation.

Traditional operating systems often require significant storage space, software installation, regular updates, and hardware resources. In contrast, StiyaOS provides a lightweight and accessible solution that allows users to use essential applications through a browser interface. This approach improves accessibility, portability, and ease of use, especially for users with low-end devices or limited storage capacity.

The system includes more than thirty pre-integrated applications such as Camera, Gallery, Image Viewer, Video Player, Music Player, Calculator, Calendar, Notes, File Explorer, Web Browser, YouTube, GitHub, VS Code Web, Android Studio Web, Weather, PDF Viewer, Settings, and other utility applications commonly found in modern operating systems. Additionally, the platform provides the capability to integrate new web applications dynamically through their web URLs, making the system flexible and extensible.

The project has been developed using React JS with Vite as the build tool to ensure fast development and optimized performance. Tailwind CSS has been used for designing a responsive and modern user interface, while Framer Motion has been utilized to create smooth animations and transitions. Zustand has been implemented for efficient global state management, enabling seamless communication among different modules and applications.

StiyaOS demonstrates how modern web technologies can be used to simulate a complete operating system environment inside a browser. The project highlights the potential of browser-based computing and provides a scalable foundation for future enhancements such as cloud storage integration, artificial intelligence assistants, collaborative applications, and advanced system customization.

The successful implementation of this project proves that web technologies are capable of delivering rich, interactive, and operating system-like experiences while maintaining accessibility, portability, and ease of deployment.

TABLE OF CONTENTS

Certificate	i
Acknowledgement	ii
Abstract	iii
Table of Contents	iv
 Chapter 1 Introduction	 1
1.1 Background of the Project	1
1.2 Need of the Project	2
1.3 Problem Statement	3
1.4 Objectives of the Project	4
1.5 Scope of the Project	5
 Chapter 2 Literature Survey	 6
2.1 Existing Systems	6
2.2 Comparative Analysis	7
2.3 Limitations of Existing Systems	8

Chapter 3 System Analysis and Requirements	9
3.1 Functional Requirements	9
3.2 Non-Functional Requirements	10
3.3 Hardware Requirements	11
3.4 Software Requirements	11
 Chapter 4 System Design and Architecture	12
4.1 System Architecture	12
4.2 Flow Diagram	13
4.3 Module Description	14
 Chapter 5 Implementation and Features	16
5.1 Technology Stack	16
5.2 Desktop Environment	17
5.3 Window Management System	18
5.4 Application Management	19
5.5 Pre-Integrated Applications	20
5.6 User Interface Design	22
 Chapter 6 Advantages, Limitations and Future Scope	24
6.1 Advantages	24
6.2 Limitations	25

6.3 Future Scope	26
Chapter 7 Conclusion	27
References	28

CHAPTER - 1

INTRODUCTION (1.1 Background of the Project)

In recent years, web technologies have evolved significantly, enabling developers to build highly interactive and feature-rich applications that can run directly within a web browser. Traditional operating systems require software installation, consume considerable storage space, and often depend on specific hardware configurations. These limitations can create challenges for users who need quick access to applications across different devices.

StiyaOS is a browser-based Operating System simulation developed to provide an operating system-like environment through a web browser. The system offers a desktop interface with multiple integrated applications and utilities that mimic the experience of a traditional operating system while eliminating the need for software installation.

The project includes more than thirty pre-integrated applications such as Camera, Gallery, Image Viewer, Video Player, Calculator, Calendar, Notes, Web Browser, YouTube, GitHub, VS Code Web, Android Studio Web, and many other useful applications. In addition, users can integrate new web applications dynamically using their URLs, making the platform flexible and expandable.

The project is developed using modern web technologies including React JS, Vite, Tailwind CSS, Framer Motion, and Zustand. These technologies provide a responsive user interface, efficient state management, and smooth animations, resulting in an engaging user experience.

StiyaOS demonstrates how modern web technologies can be utilized to create a lightweight, accessible, and scalable operating system environment that can be accessed from anywhere using a web browser.

INTRODUCTION (1.2 Need of the Project)

With the rapid growth of web technologies and cloud-based services, there is an increasing demand for applications that are accessible from anywhere without requiring installation or high-end hardware. Traditional operating systems often consume significant storage space and require users to install and update multiple applications separately. This can be inconvenient, especially for users with low-end devices or limited system resources.

The need for StiyaOS arises from the requirement for a lightweight and portable operating system environment that can run entirely within a web browser. The project enables users to access essential applications such as media players, productivity tools, development environments, and web services from a single platform without the complexity of managing software installations.

StiyaOS also addresses the problem of accessibility by allowing users to use their applications across different devices and operating systems through a browser. The ability to integrate new web applications dynamically through URLs further enhances the flexibility and usability of the system.

Therefore, the project provides a modern, user-friendly, and scalable solution that combines the features of a traditional operating system with the accessibility and convenience of web technologies.

INTRODUCTION (1.3 Problem Statement)

Traditional operating systems provide a wide range of functionalities but often require substantial storage space, software installation, regular updates, and hardware resources. Users frequently need to install multiple applications separately, which can lead to compatibility issues, increased maintenance efforts, and reduced system performance.

Moreover, accessing the same set of applications across different devices can be difficult due to platform dependencies and varying system configurations. Users with low-end devices may also face limitations in running resource-intensive software efficiently.

StiyaOS is developed to address these challenges by providing a browser-based operating system environment that offers essential applications through a unified and interactive interface. The system eliminates the need for local installation of many applications and allows users to access their tools directly from a web browser.

By integrating multiple applications and enabling the addition of new web applications through URLs, StiyaOS aims to provide a lightweight, accessible, and flexible alternative to traditional desktop environments while maintaining a familiar operating system experience.

Proposed Solution

StiyaOS is designed as a browser-based Operating System simulation that provides a desktop-like environment directly within a web browser. The system integrates more than thirty essential applications, including media players, productivity tools, development environments, and web services, allowing users to access them from a single interface.

The platform also enables users to add new web applications dynamically through URLs, making it flexible and extensible. By utilizing modern web technologies such as React JS, Vite, Tailwind CSS, Framer Motion, and Zustand, StiyaOS offers a responsive and interactive experience while reducing dependency on hardware and software installations.

Thus, the project provides a portable, user-friendly, and scalable solution that combines the familiarity of traditional operating systems with the convenience and accessibility of web technologies.

INTRODUCTION (1.4 Objectives of the Project)

The primary objective of StiyaOS is to develop a browser-based Operating System simulation that provides users with a modern, interactive, and feature-rich desktop environment accessible from any web browser. The specific objectives of the project are as follows:

- To design and develop a desktop environment that simulates the look and feel of a traditional operating system with windows, taskbar, desktop icons, and multitasking capabilities.
- To integrate more than 30 essential applications such as Photo Viewer, Media Player, Video Player, File Management System, Terminal, Camera, Calendar, Calculator, Notepad, Clock, Web Browser, YouTube, Instagram, LinkedIn, Reddit, Google Workspace applications, VS Code, and other utility applications.
- To develop an App Store module that enables users to install and manage applications within the system.
- To provide support for integrating new applications dynamically using web application URLs as well as custom React components, making the system flexible and extensible.
- To implement an AI-powered assistant capable of performing tasks such as opening applications, interacting with the chatbot, and executing user commands to improve productivity and user experience.
- To implement an efficient window management system that supports opening multiple applications simultaneously with features such as minimize, maximize, drag, resize, and close operations.
- To develop a responsive and visually appealing user interface using React JS, Tailwind CSS, and Framer Motion for smooth animations and modern design.

- To manage application states, user preferences, installed applications, and system settings efficiently using Zustand state management.
- To create a lightweight, platform-independent, and scalable system that can be accessed from any device through a web browser.
- To demonstrate the capabilities of modern web technologies in creating an operating system-like environment and provide a foundation for future enhancements and innovations.

INTRODUCTION (1.5 Scope of the Project)

The scope of StiyaOS is to develop a browser-based Operating System simulation that provides users with a modern and interactive computing environment directly through a web browser. The project focuses on delivering the essential functionalities of a traditional operating system while utilizing the flexibility and accessibility of modern web technologies.

StiyaOS aims to bridge the gap between desktop operating systems and web applications by integrating multiple applications and services within a unified platform. The system is designed to be lightweight, platform-independent, and easy to use, allowing users to access applications without the need for installation or complex system configurations.

The scope of the project includes the following features:

- Development of a Windows-like desktop environment with desktop icons, taskbar, start menu, and notification area.
- Implementation of a window management system that supports opening multiple applications simultaneously with functionalities such as drag and drop, resize, minimize, maximize, and close operations.
- Integration of more than 30 pre-installed applications including Photo Viewer, Media Player, Video Player, File Management System, Terminal, Camera, Calendar, Calculator, Notepad, Clock, Web Browser, YouTube, Instagram, LinkedIn, Reddit, Google Workspace applications, VS Code, Settings, and other utility applications.
- Development of an App Store module through which users can install and manage applications within the operating system environment.
- Support for integrating new applications dynamically using web application URLs as well as custom React components, making the platform highly flexible and extensible.
- Implementation of an AI-powered assistant that can perform tasks such as opening applications, interacting with chatbots, and executing user commands to enhance productivity and improve user experience.
- Development of a responsive and visually appealing user interface using React JS, Tailwind CSS, and Framer Motion to provide smooth animations and an engaging user experience.

- Efficient management of application states, user preferences, and system settings using Zustand state management.
- Provision of a platform-independent environment that can run on desktops, laptops, tablets, and other devices equipped with a modern web browser and internet connection.

The project is primarily focused on frontend implementation and browser-based application integration. The current version does not include advanced backend services such as user authentication, cloud synchronization, or distributed storage systems. However, the architecture of StiyaOS has been designed in such a way that these features can be integrated in future versions without major modifications.

Future enhancements of the project may include the following:

- Cloud storage and file synchronization.
- Offline support using Progressive Web App (PWA) technologies.
- Advanced AI assistant capabilities with voice interaction.
- Real-time collaboration and communication tools.
- Integration of additional productivity, educational, and development applications.
- Theme customization and personalized desktop environments.

Thus, the scope of StiyaOS extends beyond a simple web application and serves as a foundation for developing a complete browser-based computing ecosystem. The project demonstrates the capabilities of modern web technologies in creating scalable, interactive, and user-friendly operating system environments that can evolve with future technological advancements.

CHAPTER - 2

LITERATURE SURVEY

The rapid advancement of web technologies has transformed the way users interact with applications and computing environments. Modern web browsers are capable of supporting complex and interactive applications that were previously limited to desktop operating systems. As a result, several browser-based operating system environments and web desktop platforms have been developed to provide users with an operating system-like experience through the internet.

Traditional operating systems such as *Microsoft Windows, Linux, and macOS* provide a complete computing environment with multitasking capabilities, file management systems, media applications, and software installation support. However, these operating systems require local installation, regular updates, considerable storage space, and compatible hardware resources. They are also platform-dependent and may not provide the same experience across different devices.

With the growth of cloud computing, browser-based operating systems such as Chrome OS introduced the concept of running applications primarily through a web browser. Chrome OS provides a lightweight environment and cloud integration, but its application ecosystem and customization capabilities are comparatively limited when compared to a traditional desktop operating system.

Several web desktop projects and browser-based environments have also been developed to simulate desktop operating systems. These systems provide features such as desktop interfaces, window management, and web application integration. However, many existing solutions focus only on visual simulation and lack advanced functionalities such as dynamic application integration, AI assistance, extensible app stores, and modern user interfaces.

StiyaOS has been developed by studying these existing systems and identifying their limitations. The project combines the accessibility of browser-based applications with the familiarity of traditional operating systems.

It provides a Windows-like desktop environment with multitasking support, integrated applications, an App Store, AI-powered assistance, and the capability to add new applications dynamically through web URLs and React components.

The project leverages modern technologies such as React JS, Vite, Tailwind CSS, Framer Motion, and Zustand to create a responsive, interactive, and scalable operating system simulation. Unlike many existing browser-based desktop environments, StiyaOS emphasizes extensibility, user experience, and intelligent assistance, making it a modern solution for browser-based computing.

Therefore, the literature survey indicates that while existing operating systems and browser-based environments provide valuable features, there remains significant scope for developing a more flexible, intelligent, and extensible web-based operating system. StiyaOS aims to address these requirements and contribute to the evolution of browser-based computing systems.

LITERATURE SURVEY (2.1 Existing Systems)

Various operating systems and browser-based desktop environments have been developed over the years to provide users with a convenient and efficient computing experience. These systems offer different features and functionalities, but they also possess certain limitations that create opportunities for further improvement.

1. Traditional Desktop Operating Systems

Traditional operating systems such as Windows, Linux, and macOS provide a complete computing environment with support for multitasking, file management, software installation, and hardware integration. These operating systems are widely used due to their stability and extensive application ecosystems.

However, traditional operating systems require software installation, regular updates, considerable storage space, and compatible hardware resources. Users also need to manage multiple applications separately, which can increase system complexity and maintenance efforts.

2. Chrome OS

Chrome OS is a browser-centric operating system developed to provide a lightweight and cloud-based computing experience. It allows users to access web applications directly and offers seamless integration with cloud services.

Although Chrome OS is efficient and lightweight, its functionality is primarily dependent on internet connectivity and cloud services. In addition, customization options and support for third-party applications are comparatively limited when compared to traditional operating systems.

3. Browser-Based Desktop Environments

Several browser-based desktop environments have been developed to simulate operating system interfaces within web browsers. These platforms generally provide desktop icons, window systems, and basic application integration.

Most of these systems focus mainly on visual simulation and lack advanced functionalities such as AI assistance, dynamic application installation, app stores, and comprehensive application ecosystems. Their scalability and flexibility are often limited, reducing their usability for practical computing tasks.

4. StiyaOS

StiyaOS is a browser-based Operating System simulation designed to overcome many of the limitations found in existing systems. It provides a Windows-like user interface with desktop icons, taskbar, multitasking support, and window management capabilities.

The system integrates more than 30 applications, including media players, productivity tools, development environments, social media applications, and utility software. It also features an App Store that allows users to install applications and supports dynamic integration of new applications through web URLs and React components.

In addition, StiyaOS incorporates an AI-powered assistant that can perform tasks such as opening applications, interacting with chatbots, and executing user commands. These features make StiyaOS more flexible, intelligent, and extensible than many existing browser-based desktop environments.

LITERATURE SURVEY (2.2 Comparative Analysis)

A comparative analysis of existing operating systems and browser-based desktop environments is essential to understand their capabilities and limitations. The comparison below highlights the key differences between Traditional Operating Systems, Chrome OS, Browser-Based Desktop Environments, and StiyaOS.

Feature	Traditional OS	Chrome OS	BDE	StiyaOS
Installation Required	Yes	Yes	No	No
Platform Independent	No	Limited	Yes	Yes
Browser-Based Access	No	Partial	Yes	Yes
Desktop Interface	Yes	Yes	Yes	Yes
Window Management	Yes	Yes	Limited	Yes
Pre-Integrated Applications	Yes	Limited	Limited	30+ Applications
Dynamic App Integration	No	Limited	Limited	Yes
App Store Support	Yes	Yes	Rare	Yes
AI Assistant	Limited	Limited	No	Yes
Custom React Components	No	No	No	Yes
Modern UI and Animations	Moderate	Moderate	Moderate	Advanced
Accessibility Across Devices	Limited	Good	Good	Excellent
Storage Requirement	High	Medium	Low	Low

From the above comparison, it can be observed that Traditional Operating Systems provide a rich computing environment but require installation, regular maintenance, and considerable

system resources. Chrome OS offers a lightweight and cloud-based experience but provides limited flexibility for customization and application integration.

Browser-Based Desktop Environments provide accessibility and portability but generally focus on visual simulation rather than practical usability. Most of these systems do not offer features such as AI assistance, dynamic application installation, or a dedicated App Store.

StiyaOS combines the advantages of traditional operating systems and browser-based environments while minimizing their limitations. It offers a Windows-like interface, integrated applications, an AI-powered assistant, dynamic application integration, and modern user experience features. Therefore, StiyaOS provides a more flexible, intelligent, and scalable browser-based computing platform.

LITERATURE SURVEY (2.3 Limitations of Existing Systems)

Although various operating systems and browser-based desktop environments are available today, they suffer from several limitations that affect their usability, accessibility, and flexibility.

Traditional operating systems such as Windows, Linux, and macOS require users to install applications locally on their devices. This increases storage consumption and often demands regular software updates and maintenance. In addition, these operating systems are hardware-dependent and may not perform efficiently on low-end devices.

Chrome OS provides a lightweight and cloud-oriented computing experience; however, its functionality heavily depends on internet connectivity and cloud services. The operating system offers limited customization options and does not provide complete flexibility for integrating third-party applications according to user requirements.

Most browser-based desktop environments are developed primarily to imitate the visual appearance of an operating system rather than providing a complete operating system experience. They often lack important features such as:

- AI-powered assistance for automating user tasks.
- Dynamic installation of applications through web URLs.
- Support for integrating custom React components as applications.
- Dedicated App Store for installing and managing applications.
- Advanced window management features such as multitasking, drag-and-drop, and window resizing.
- Smooth animations and modern user interface design.
- Efficient global state management for handling multiple applications simultaneously.

Another significant limitation of existing systems is the lack of personalization and extensibility. Many platforms do not allow users to customize their workspace or extend the functionality of the system according to their needs.

StiyaOS has been developed to overcome these limitations by providing a browser-based operating system that combines a modern desktop interface, AI-powered assistance, dynamic application integration, an App Store, and more than 30 pre-integrated applications. The system focuses on accessibility, flexibility, and scalability while delivering a familiar operating system experience through a web browser.

Thus, the limitations of existing systems highlight the need for a more intelligent, extensible, and user-friendly browser-based operating system, which forms the foundation of the StiyaOS project.

CHAPTER - 3

SYSTEM ANALYSIS AND REQUIREMENTS

(3.1 Functional Requirements)

Functional requirements define the major functionalities and services provided by StiyaOS. The system is designed to offer an operating system-like experience within a web browser while supporting multiple integrated applications and user interactions.

The functional requirements of StiyaOS are as follows:

- The system shall provide a desktop environment with desktop icons, taskbar, start menu, and notification area.
- The system shall allow users to open, close, minimize, maximize, drag, and resize multiple application windows.
- The system shall support multitasking by allowing multiple applications to run simultaneously.
- The system shall provide more than 30 pre-integrated applications, including Photo Viewer, Media Player, Video Player, File Manager, Terminal, Camera, Calculator, Calendar, Notepad, Clock, Web Browser, YouTube, Instagram, LinkedIn, Reddit, Google Workspace applications, VS Code, and Settings.
- The system shall provide an App Store for browsing and installing applications.
- The system shall allow users to install new applications dynamically using web application URLs.
- The system shall support integration of custom React components as applications.
- The system shall provide an AI-powered assistant capable of opening applications and performing user-defined tasks.
- The system shall maintain application states and user preferences using Zustand state management.
- The system shall provide responsive layouts to support different screen sizes and devices.

The above requirements ensure that StiyaOS provides a complete and interactive browser-based operating system environment while maintaining flexibility and scalability.

SYSTEM ANALYSIS AND REQUIREMENTS

(3.2 Non-Functional Requirements)

Non-functional requirements define the quality attributes and operational characteristics of StiyaOS. These requirements ensure that the system performs efficiently and provides a satisfactory user experience.

The non-functional requirements of StiyaOS are as follows:

- Performance

The system should load quickly and provide smooth interaction while opening or switching between applications. Animations and transitions should execute efficiently without affecting the overall performance of the system.

- Usability

The user interface should be simple, intuitive, and easy to use. Users should be able to access applications, manage windows, and perform tasks without requiring extensive technical knowledge.

- Reliability

The system should function consistently under normal operating conditions. Integrated applications and system components should operate without frequent crashes or unexpected behavior.

- Scalability

The architecture of StiyaOS should support the addition of new applications and features without requiring major changes to the existing system. The App Store and dynamic application integration should allow future expansion of the platform.

- Portability

The system should be platform-independent and capable of running on different operating systems such as Windows, Linux, and macOS through a modern web browser.

- Maintainability

The project should be developed using modular and reusable components so that future modifications, updates, and bug fixes can be implemented easily.

- Responsiveness

The user interface should adapt to different screen sizes and resolutions, allowing users to access the system on desktops, laptops, tablets, and other compatible devices.

- Security

The system should ensure safe handling of user data and integrated web applications. Only trusted URLs and applications should be allowed to prevent unauthorized or harmful content.

These non-functional requirements ensure that StiyaOS remains efficient, user-friendly, scalable, and reliable while providing an operating system-like experience within a web browser.

SYSTEM ANALYSIS AND REQUIREMENTS

(3.3 Hardware Requirements)

The hardware requirements specify the minimum and recommended system configurations required to run StiyaOS efficiently. Since StiyaOS is a browser-based Operating System simulation, it does not require high-end hardware and can run on most modern computing devices.

Minimum Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB
- Storage Space: 500 MB free space
- Display Resolution: 1366 × 768 pixels
- Internet Connection: Stable broadband connection
- Input Devices: Keyboard and Mouse or Touchpad

Recommended Hardware Requirements

- Processor: Intel Core i5 or above
- RAM: 8 GB or more
- Storage Space: 1 GB free space
- Display Resolution: 1920 × 1080 pixels
- Internet Connection: High-speed broadband connection
- Input Devices: Keyboard, Mouse, or Touchscreen

The above hardware configuration ensures smooth execution of the operating system interface, integrated applications, animations, and AI-powered functionalities. Since the project runs inside a web browser, it remains lightweight and can be accessed from a wide range of devices without requiring specialized hardware.

SYSTEM ANALYSIS AND REQUIREMENTS

(3.4 Software Requirements)

The software requirements specify the technologies, tools, and software platforms used for the development and execution of StiyaOS. The project has been developed using modern web technologies to ensure high performance, scalability, and a responsive user experience.

Development Technologies

- Frontend Framework: React JS
- Build Tool: Vite
- Programming Language: JavaScript (ES6+)
- Styling Framework: Tailwind CSS
- State Management: Zustand
- Animation Library: Framer Motion
- Fonts: Google Fonts
- Markup Language: HTML5
- Styling Language: CSS3

Development Tools

- Code Editor: Visual Studio Code
- Version Control System: Git
- Repository Hosting: GitHub
- Package Manager: npm
- Browser Developer Tools: Chrome DevTools

Web APIs Used

- MediaDevices API for Camera access
- Local Storage API for storing user preferences and settings
- File API for file handling and file management
- Clipboard API for copy and paste operations
- Fullscreen API for fullscreen mode

- Web Audio API for audio playback and media handling

Browser Requirements

- Google Chrome version 100 or above
- Microsoft Edge version 100 or above
- Mozilla Firefox version 100 or above
- Any modern browser supporting HTML5, CSS3, and JavaScript ES6+

Operating System Support

- Windows 10 or above
- Linux-based operating systems
- macOS
- Android and iPadOS with modern browsers

The above software requirements ensure that StiyaOS can be developed, maintained, and executed efficiently across different platforms while providing a consistent and interactive operating system-like experience within a web browser.

CHAPTER - 4

SYSTEM DESIGN AND ARCHITECTURE

(4.1 System Architecture)

The architecture of StiyaOS is designed using a modular approach to ensure scalability, maintainability, and efficient management of different functionalities. The system follows a component-based architecture where each module performs a specific task while interacting seamlessly with other modules.

The architecture consists of the following major layers:

1. User Interface Layer

This layer is responsible for providing the graphical user interface of StiyaOS. It includes desktop icons, taskbar, start menu, notification panel, application windows, and system dialogs. The interface is designed using React JS and Tailwind CSS, while Framer Motion is used to create smooth animations and transitions.

2. Window Management Layer

This layer manages all application windows within the system. It handles functionalities such as:

- Opening and closing windows
- Dragging windows across the desktop
- Resizing application windows
- Minimizing and maximizing windows
- Managing multiple applications simultaneously

The window management system provides users with a traditional desktop-like experience and enables multitasking within the browser.

3. Application Management Layer

This layer manages all applications available in StiyaOS. It includes:

- Pre-integrated applications
- Application launching mechanism
- App Store
- Installation and uninstallation of applications
- Dynamic application integration through web URLs
- Custom React component integration

The Application Management Layer ensures that users can easily access and manage applications according to their requirements.

4. AI Assistant Layer

This layer provides intelligent assistance to users. The AI Assistant is capable of:

- Opening applications through commands
- Performing predefined tasks
- Interacting with chatbots
- Helping users navigate the operating system

This feature enhances productivity and provides a more interactive user experience.

5. State Management Layer

The State Management Layer is implemented using Zustand. It manages:

- Opened applications
- Window positions and sizes
- User preferences
- System settings
- Installed applications
- Theme and interface configurations

This layer ensures efficient communication among different components of the system.

6. External Services Layer

This layer consists of external web applications and services integrated into StiyaOS, such as:

- YouTube
- Instagram
- LinkedIn
- Reddit
- Google Workspace Applications
- GitHub
- VS Code
- Other web applications integrated through URLs

The External Services Layer expands the capabilities of StiyaOS and allows users to access multiple services from a unified platform.

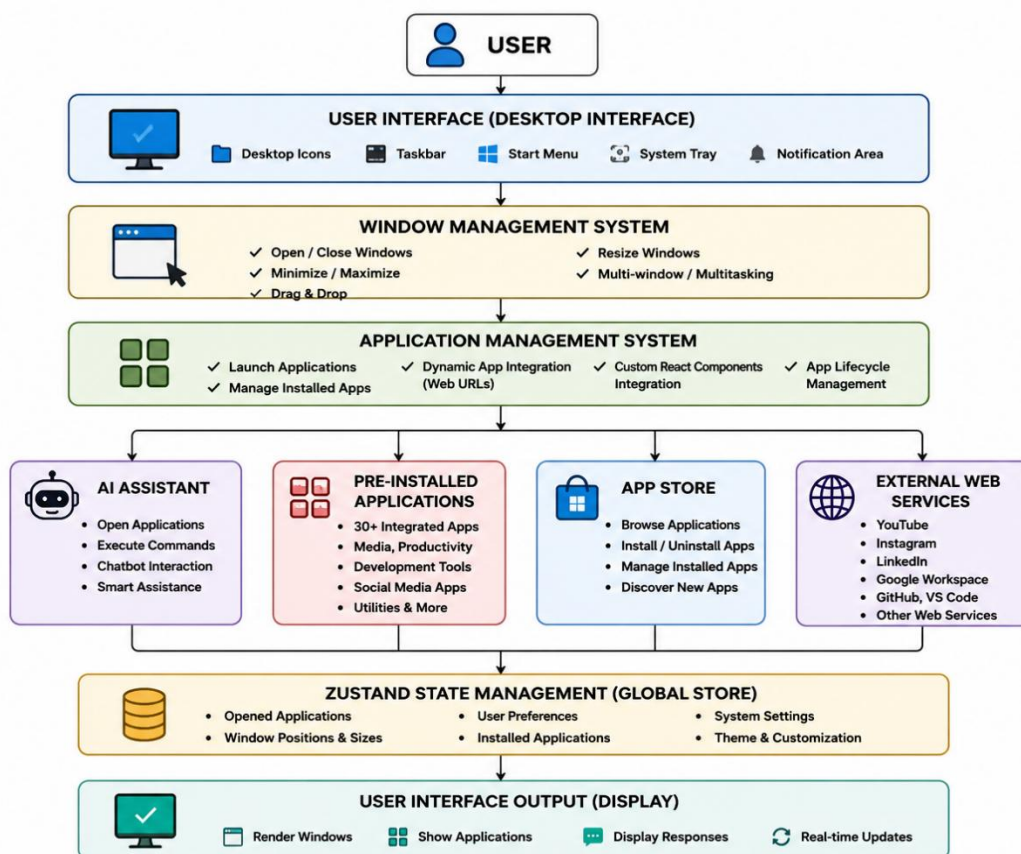
Thus, the modular architecture of StiyaOS provides flexibility, scalability, and ease of maintenance while delivering a complete browser-based operating system experience.

SYSTEM DESIGN AND ARCHITECTURE

(4.2 Flow Diagram)

The working flow of StiyaOS describes how different modules interact with each other to provide a seamless operating system-like experience within a web browser. The system begins with user interaction and processes requests through various layers before delivering the desired output.

The flow of the system is as follows:



Working Flow:

1. The user interacts with the Desktop Interface through desktop icons, taskbar, start menu, or keyboard shortcuts.
2. The Window Management System receives the request and manages operations such as opening, closing, resizing, minimizing, or maximizing application windows.

3. The Application Management System determines whether the requested application is:
4. A pre-installed application
5. An application installed from the App Store
6. A web application integrated through a URL
7. A custom React component
8. If the request involves intelligent assistance, the AI Assistant processes the command and performs the required task, such as opening an application or interacting with the chatbot.
9. Zustand State Management stores and updates application states, user preferences, window positions, installed applications, and system settings.
10. Finally, the processed data is rendered on the user interface and displayed to the user in real time.

The flow diagram demonstrates that StiyaOS follows a modular and event-driven architecture, where each component performs a dedicated task while maintaining efficient communication with other modules. This architecture improves scalability, maintainability, and overall user experience.

SYSTEM DESIGN AND ARCHITECTURE

(4.3 Module Description)

StiyaOS is developed using a modular architecture in which each module is responsible for a specific functionality. This approach improves maintainability, scalability, and overall system performance. The major modules of the system are described below.

1. Desktop Environment Module

The Desktop Environment Module is responsible for providing the primary user interface of StiyaOS. It creates a Windows-like environment consisting of desktop icons, taskbar, start menu, wallpapers, and notification panels. This module serves as the entry point for all user interactions.

Main Functions:

- Display desktop icons and shortcuts.
- Provide taskbar and start menu.
- Manage wallpapers and themes.
- Display notifications and system status.

2. Window Management Module

The Window Management Module manages all application windows within the operating system. It enables users to run multiple applications simultaneously and provides a smooth multitasking experience.

Main Functions:

- Open and close application windows.
- Minimize and maximize windows.
- Resize and drag windows.
- Maintain window focus and stacking order.
- Support multitasking.

3. Application Management Module

This module is responsible for managing all applications available in StiyaOS. It handles pre-installed applications, applications installed from the App Store, and applications integrated through URLs or React components.

Main Functions:

- Launch and terminate applications.
- Manage pre-installed applications.
- Support custom React component integration.
- Integrate web applications using URLs.
- Maintain installed application records.

4. App Store Module

The App Store Module allows users to browse, install, and manage applications within StiyaOS. It provides a centralized platform for expanding the capabilities of the operating system.

Main Functions:

- Display available applications.
- Install and uninstall applications.
- Search applications.
- Manage application updates.

5. AI Assistant Module

The AI Assistant Module provides intelligent assistance to users by understanding commands and performing various tasks automatically.

Main Functions:

- Open applications through commands.
- Interact with chatbots.
- Perform predefined system tasks.
- Assist users in navigating the operating system.

6. State Management Module

The State Management Module is implemented using Zustand and is responsible for managing global states across the application.

Main Functions:

- Store application states.
- Manage window positions and sizes.
- Store user preferences.
- Manage themes and settings.
- Synchronize data across components.

7. External Services Module

This module handles integration with external web applications and services that are accessible within StiyaOS.

Integrated Services Include:

- YouTube
- Instagram
- LinkedIn
- Reddit
- Google Workspace Applications
- GitHub
- VS Code
- Other web applications integrated through URLs

The modular design of StiyaOS ensures that each component performs a dedicated function while maintaining smooth interaction with other modules. This architecture makes the system flexible, scalable, and easy to maintain, allowing future enhancements without major modifications to the existing system.

CHAPTER - 5

IMPLEMENTATION AND FEATURES (5.1 Technology Stack)

StiyaOS is developed using modern web technologies to provide a responsive, scalable, and interactive operating system-like experience within a web browser. The selected technologies ensure high performance, efficient state management, and a visually appealing user interface.

Frontend Technologies

- React JS

React JS is used as the primary frontend library for developing reusable and interactive user interface components. It enables efficient rendering and improves the maintainability of the application.

- Vite

Vite is used as the build tool for the project. It provides faster development server startup, instant hot module replacement, and optimized production builds, making the development process efficient.

- Tailwind CSS

Tailwind CSS is used for styling the user interface. It offers utility-first CSS classes that help in creating responsive layouts and modern designs with minimal custom CSS.

- Framer Motion

Framer Motion is used to implement animations and transitions throughout the application. It provides smooth window opening and closing animations, hover effects, and interactive user experiences.

- Zustand

Zustand is used for global state management. It manages application states such as opened applications, installed applications, user preferences, window positions, themes, and other system settings efficiently.

Programming and Markup Languages

- JavaScript (ES6+)

JavaScript is used to implement the business logic and interactivity of the application.

- HTML5

HTML5 is used to structure the web pages and application components.

- CSS3

CSS3 is used along with Tailwind CSS to enhance the styling and appearance of the system.

Fonts and Icons

- Google Fonts are used to provide modern typography and improve readability.
- Modern icon libraries are integrated to create visually appealing application icons and interface elements.

Development Tools

- Visual Studio Code is used as the primary code editor.
- Git is used for version control.
- GitHub is used for source code management and collaboration.
- npm is used as the package manager for installing dependencies and libraries.

The combination of these technologies enables StiyaOS to deliver a modern, lightweight, and scalable browser-based operating system environment with excellent performance and user experience.

IMPLEMENTATION AND FEATURES (5.2 Desktop Environment)

The Desktop Environment is the central component of StiyaOS and serves as the primary interface through which users interact with the system. It is designed to provide a familiar Windows-like experience while utilizing the flexibility of modern web technologies.

The desktop consists of wallpapers, desktop icons, taskbar, start menu, notification panel, and multiple application windows. The interface is responsive and adapts to different screen sizes, allowing users to access the operating system from various devices.

The major features of the Desktop Environment are as follows:

- Windows-like User Interface

StiyaOS provides a modern desktop interface inspired by traditional operating systems. Users can interact with applications using desktop icons and manage tasks through an intuitive taskbar and start menu.

- Desktop Icons

The desktop contains shortcuts for frequently used applications and utilities. Users can launch applications directly by clicking on their respective icons.

- Taskbar

The taskbar displays currently running applications and provides quick access to the start menu, search functionality, notifications, and system settings. It also helps users switch between multiple applications efficiently.

- Start Menu

The start menu provides categorized access to all installed applications and system utilities. Users can search for applications, launch them, and access important system features from a single location.

- Notification Panel

The notification panel displays important information such as system alerts, reminders, and application notifications, improving the overall user experience.

- Multiple Window Support

Users can open multiple applications simultaneously. Each application runs inside an independent window that can be moved freely across the desktop.

- Responsive Design

The desktop environment is fully responsive and can adapt to different screen resolutions and device sizes, ensuring a consistent experience across desktops, laptops, and tablets.

- Themes and Customization

The system supports themes, wallpapers, and user preferences, allowing users to personalize the appearance of the operating system according to their needs.

The Desktop Environment of StiyaOS is developed using React JS and Tailwind CSS, while Framer Motion is used to implement smooth animations and transitions. Zustand is utilized for managing desktop states, open windows, and user preferences.

Thus, the Desktop Environment provides a modern, interactive, and user-friendly operating system experience directly within a web browser while maintaining flexibility and scalability.

IMPLEMENTATION AND FEATURES

(5.3 Window Management System)

The Window Management System is one of the most important components of StiyaOS. It is responsible for managing all application windows and providing users with a multitasking experience similar to traditional desktop operating systems. This module ensures that users can open, organize, and interact with multiple applications simultaneously in a convenient and efficient manner.

The system is designed to mimic the behavior of modern operating systems by allowing application windows to operate independently while maintaining smooth interaction with the desktop environment.

The major features of the Window Management System are as follows:

- Opening and Closing Windows

Users can launch applications from the desktop, start menu, or App Store. Each application opens in a separate window that can be closed independently without affecting other running applications.

- Minimize and Maximize Operations

The system allows users to minimize application windows to the taskbar and restore them whenever required. Users can also maximize windows to utilize the entire screen area for better productivity.

- Drag and Drop Support

Application windows can be moved freely across the desktop using drag-and-drop functionality. This enables users to organize their workspace according to their preferences.

- Window Resizing

Users can resize application windows dynamically by dragging their borders. This feature allows efficient utilization of screen space and improves multitasking capabilities.

- Multi-Window Support

StiyaOS supports running multiple applications simultaneously. Users can switch between applications easily without losing their current progress or workspace.

- Window Focus Management

The system maintains the focus state of windows and ensures that the active window remains on top of other opened windows. This helps users interact with the desired application without confusion.

- Smooth Animations

Opening, closing, minimizing, and maximizing windows are accompanied by smooth animations implemented using Framer Motion. These animations enhance the visual appeal and provide a modern user experience.

Implementation Details

The Window Management System is implemented using React JS and Zustand. React components are used to render application windows dynamically, while Zustand manages the global state of opened applications, window positions, sizes, and active window status.

Framer Motion is integrated to provide smooth transitions and animations during window operations. The modular design of the system allows new applications and window functionalities to be added easily without affecting the existing architecture.

Thus, the Window Management System plays a crucial role in delivering an interactive and operating system-like experience, making StiyaOS more flexible, user-friendly, and efficient.

IMPLEMENTATION AND FEATURES

(5.4 Application Management System)

The Application Management System is responsible for managing all applications available in StiyaOS. It provides users with a centralized platform to launch, install, organize, and interact with applications seamlessly. The module is designed to support both pre-installed applications and dynamically integrated applications, making the system highly flexible and extensible.

StiyaOS comes with more than 30 pre-integrated applications that cover entertainment, productivity, development, communication, and social networking requirements. The Application Management System ensures smooth interaction among these applications while maintaining efficient system performance.

The major features of the Application Management System are as follows:

- Pre-Integrated Applications

StiyaOS includes a wide range of pre-installed applications such as Photo Viewer, Media Player, Video Player, File Management System, Terminal, Camera, Calendar, Calculator,

Clock, Notepad, Settings, Web Browser, YouTube, Instagram, LinkedIn, Reddit, GitHub, VS Code, Google Workspace applications, and many other utility applications.

- Dynamic Application Integration

The system allows users to integrate web applications dynamically by providing their URLs. Once integrated, these applications behave like native applications within StiyaOS and can be accessed through desktop icons, the start menu, or the App Store.

- Custom React Component Support

Developers can integrate applications directly as React components. This feature enables the development of customized applications that can interact closely with the operating system environment.

- App Store

StiyaOS includes an App Store that allows users to browse, install, and manage applications. The App Store acts as a centralized repository for extending the functionality of the operating system.

- Application Search

Users can search for applications quickly using the search functionality integrated into the Start Menu and App Store. This improves accessibility and saves time.

- Application State Persistence

The system maintains the states of running applications using Zustand. User preferences, installed applications, and window states are preserved to provide a smooth and consistent experience.

- Categorization of Applications

Applications are organized into categories such as:

- Productivity Applications
- Media and Entertainment Applications

- Social Media Applications
- Development Tools
- Utility Applications
- AI and Smart Tools

Implementation Details

The Application Management System is developed using React JS and Zustand. React components are used to render applications dynamically, while Zustand manages installed applications, running application states, and application configurations.

The modular design of this system allows new applications to be added easily without affecting existing functionalities. This makes StiyaOS scalable and capable of supporting future enhancements.

Thus, the Application Management System provides a flexible, efficient, and user-friendly platform for managing applications and forms one of the core components of StiyaOS.

IMPLEMENTATION AND FEATURES

(5.5 Pre-Integrated Applications)

StiyaOS comes with more than 30 pre-integrated applications that provide users with a complete operating system-like experience within a web browser. These applications are categorized into media, productivity, social networking, development, utility, and AI-based applications to meet different user requirements.

The major pre-integrated applications available in StiyaOS are listed below:

S. No.	Application Name	Purpose
1	Photo Viewer	View and manage images
2	Gallery	Browse image collections
3	Media Player	Play audio and video files

4	Video Player	Play videos with playback controls
5	File Management System	Manage files and folders
6	Terminal	Execute terminal-like commands
7	Camera	Capture photos and videos
8	Calculator	Perform maths calculations
9	Calendar	Manage dates and events
10	Clock	Display time and alarms
11	Notepad	Create and edit text notes
12	Settings	Configure system preferences
13	Web Browser	Browse websites within StiyaOS
14	App Store	Install and manage applications
15	AI Assistant	Perform tasks and provide assistance
16	Youtube	Watch videos and entertainment content
17	Instagram	Access social media platform
18	Linkedin	Professional networking
19	Reddit	Community discussions and forums
20	Google Drive	Cloud file storage
21	Gmail	Email communication
22	Google Drive	Document editing
23	Google Docs	Spreadsheet management
24	Google Slides	Presentation creation
25	Google Maps	Navigation and location services
26	GitHub	Source code hosting and collaboration
27	VS Code	Online code editing
28	Weather	Display weather information
29	PDF Viewer	View PDF documents
30	Music Player	Play audio and music files
31	QR Scanner	Scan QR codes
32	Voice Recorder	Record audio
33	Sticky Notes	Quick note taking
34	Image Editor	Edit images and photos
35	ChatBot	AI-based conversational assistant

Apart from these applications, StiyaOS also supports the integration of new applications dynamically. Users can add web applications by providing their URLs or integrate custom React components, enabling the system to expand according to user requirements.

The availability of a diverse range of applications makes StiyaOS a versatile and user-friendly browser-based operating system. The modular architecture ensures that new applications can be incorporated in future versions without affecting the existing system functionalities.

IMPLEMENTATION AND FEATURES

(5.6 User Interface Design)

The User Interface (UI) of StiyaOS is designed to provide a modern, intuitive, and visually appealing operating system experience within a web browser. The interface is inspired by traditional desktop operating systems while incorporating modern web design principles to enhance usability and accessibility.

The UI is developed using React JS and Tailwind CSS, while Framer Motion is utilized to implement smooth animations and transitions. The design focuses on simplicity, responsiveness, and ease of interaction so that users can perform tasks efficiently.

The major components of the User Interface are as follows:

- Desktop Screen

The desktop serves as the primary workspace of StiyaOS. It displays wallpapers, application icons, and shortcuts that allow users to launch applications quickly. The desktop is designed to resemble a traditional operating system, providing familiarity to users.

- Taskbar

The taskbar is positioned at the bottom of the screen and provides quick access to the Start Menu, running applications, notifications, and system settings. It helps users switch between applications efficiently and improves multitasking capabilities.

- Start Menu

The Start Menu acts as a centralized hub for accessing all applications and system features. It provides categorized lists of applications along with search functionality for quick navigation.

- Application Windows

Every application runs within an independent window. These windows support operations such as opening, closing, minimizing, maximizing, dragging, and resizing. The window-based design enables users to run multiple applications simultaneously.

- App Store Interface

The App Store provides an organized interface for browsing, searching, and installing applications. Users can expand the functionality of StiyaOS by adding new applications according to their requirements.

- AI Assistant Interface

The AI Assistant provides a conversational interface through which users can interact with the system. It helps users perform tasks such as opening applications, executing commands, and obtaining assistance efficiently.

- Responsive Design

The user interface is fully responsive and adapts to different screen sizes and resolutions. This enables users to access StiyaOS from desktops, laptops, tablets, and other compatible devices without compromising usability.

- Smooth Animations

Framer Motion is used to implement smooth transitions and animations for window operations, menu interactions, and application launching. These animations improve the visual appeal of the system and create a more engaging user experience.

The User Interface of StiyaOS combines functionality with aesthetics to create an interactive and user-friendly operating system environment. Its modern design and responsive nature make it suitable for a wide range of users and devices while maintaining the familiarity of traditional operating systems.

CHAPTER - 6

ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

(6.1 Advantages)

StiyaOS offers several advantages over traditional operating systems and browser-based desktop environments. The major advantages of the system are as follows:

- **Browser-Based Accessibility**

StiyaOS can be accessed directly through a web browser without requiring installation, making it easy to use on different devices and platforms.

- **Lightweight Architecture**

The system consumes fewer hardware resources compared to traditional operating systems, making it suitable for low-end devices.

- **Windows-Like User Experience**

The desktop interface, taskbar, start menu, and window management system provide users with a familiar and intuitive operating system experience.

- **Rich Application Ecosystem**

StiyaOS includes more than 30 pre-integrated applications covering productivity, entertainment, social networking, development, and utility purposes.

- **Dynamic Application Integration**

Users can integrate new web applications using URLs or custom React components, allowing the system to expand according to user needs.

- **AI-Powered Assistant**

The AI Assistant helps users perform tasks efficiently by opening applications, executing commands, and interacting through a conversational interface.

- Modern User Interface

The system provides smooth animations, responsive layouts, and an attractive design that enhances user experience.

- Platform Independence

StiyaOS can run on various operating systems such as Windows, Linux, macOS, and mobile devices with modern web browsers.

- Scalability

The modular architecture allows future enhancements and easy integration of new features without major modifications.

ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

(6.2 Limitations)

Although StiyaOS provides a modern and interactive browser-based operating system experience, the current version has certain limitations that can be improved in future releases.

The major limitations of the system are as follows:

- Dependence on Internet Connectivity

Since many applications and services are web-based, a stable internet connection is required for accessing all features efficiently. Limited or no internet connectivity may restrict the functionality of some applications.

- Limited Native Hardware Access

Being a browser-based application, StiyaOS has restricted access to certain hardware resources and system-level functionalities compared to traditional operating systems.

- Browser Compatibility Issues

Although the system is designed to work on modern browsers, some features may behave differently depending on browser support and updates.

- Limited Offline Functionality

The current version primarily focuses on online services and web applications. Offline access to applications and data is limited.

- Security Concerns with External Applications

Applications integrated through external web URLs may have varying levels of security and reliability. Therefore, users should integrate only trusted applications and services.

- Resource Consumption with Multiple Applications

Opening a large number of applications simultaneously may increase browser memory usage and affect performance on low-end devices.

- Absence of Advanced Backend Services

The current implementation does not include features such as cloud synchronization, user authentication, file backup, or multi-user support.

Despite these limitations, StiyaOS successfully demonstrates the potential of modern web technologies in creating an operating system-like environment within a browser. The modular architecture of the system provides sufficient flexibility for overcoming these limitations in future versions.

ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

(6.3 Future Scope)

StiyaOS has been designed with a modular and scalable architecture, allowing new features and functionalities to be integrated in future versions. The project provides a strong foundation for developing a complete browser-based computing environment capable of competing with traditional operating systems in terms of usability and flexibility.

The future scope of the project includes the following enhancements:

- Advanced AI Assistant

The AI Assistant can be enhanced with voice recognition and natural language processing capabilities to enable voice-controlled operations and more intelligent task automation.

- User Authentication and Profiles

User registration, login, and personalized profiles can be implemented to provide a customized experience and secure access to user data.

- Cloud Storage Integration

Cloud storage services can be integrated to allow users to store, synchronize, and access files from anywhere.

- Progressive Web App (PWA) Support

StiyaOS can be converted into a Progressive Web App to provide offline support, faster loading times, and installation directly from the browser.

- Real-Time Collaboration

Features such as collaborative document editing, chat systems, and shared workspaces can be integrated to improve productivity and teamwork.

- Plugin and Extension Support

A plugin architecture can be developed to allow third-party developers to create extensions and enhance the capabilities of the operating system.

- Advanced File Management

The File Management System can be extended with cloud synchronization, file compression, encryption, and advanced search functionalities.

- Theme and Personalization

Additional themes, wallpapers, widgets, and customization options can be introduced to provide a personalized user experience.

- More Integrated Applications

The number of integrated applications can be increased to include online IDEs, video conferencing tools, educational platforms, gaming applications, and business productivity tools.

In conclusion, StiyaOS has significant potential for future growth and innovation. With continuous development and integration of emerging technologies, it can evolve into a comprehensive browser-based operating system that offers a powerful, intelligent, and user-friendly computing experience.

CHAPTER - 7

CONCLUSION

StiyaOS is a browser-based Operating System simulation developed to provide a modern, interactive, and user-friendly computing environment within a web browser. The project successfully demonstrates how modern web technologies can be utilized to create an operating system-like experience without requiring traditional software installation or high-end hardware resources.

The system provides a Windows-like user interface consisting of a desktop environment, taskbar, start menu, window management system, and multitasking capabilities. More than 30 pre-integrated applications have been incorporated into the platform, including media applications, productivity tools, development environments, social media platforms, and utility applications. In addition, the App Store and dynamic application integration through web URLs and React components make the system flexible and extensible.

The integration of an AI-powered assistant further enhances the capabilities of StiyaOS by enabling intelligent interactions and automated task execution. Modern technologies such as React JS, Vite, Tailwind CSS, Framer Motion, and Zustand have been effectively utilized to ensure responsiveness, smooth animations, efficient state management, and an attractive user interface.

The project successfully achieves its objectives of creating a lightweight, platform-independent, and scalable browser-based operating system. It also highlights the growing potential of web technologies in transforming the way users interact with computing environments.

In the future, StiyaOS can be enhanced with advanced AI capabilities, cloud storage, real-time collaboration, Progressive Web App support, and many other innovative features. Therefore, StiyaOS not only serves as an academic project but also demonstrates the possibilities of developing intelligent and feature-rich operating systems using modern web technologies.

REFERENCES

- React JS Official Documentation
<https://react.dev/>
- Vite Official Documentation
<https://vitejs.dev/>
- Tailwind CSS Official Documentation
<https://tailwindcss.com/docs>
- Zustand Official Documentation
<https://zustand.docs.pmnd.rs/>
- Framer Motion Official Documentation
<https://www.framer.com/motion/>
- Mozilla Developer Network (MDN) Web Docs
<https://developer.mozilla.org/>
- Google Fonts Documentation
<https://fonts.google.com/>
- HTML5 Specification
<https://html.spec.whatwg.org/>
- CSS3 Documentation
<https://www.w3.org/Style/CSS/>
- JavaScript Documentation
<https://developer.mozilla.org/en-US/docs/Web/JavaScript>